

本节内容

无符号整数的 除法运算原理

知识总览

无符号整数除法

无符号整数除法的手算方法

除法电路的基本结构、基本原理

穿插介绍：除法异常

“除数为0”异常

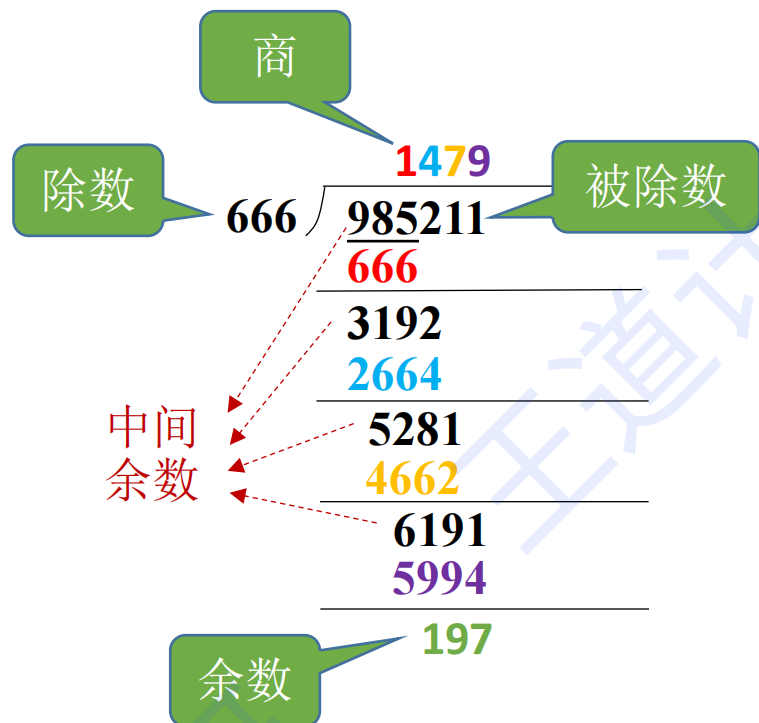
“商溢出”异常

对于除法电路，建议**重点关注**：除法电路的**开始状态**、**结束状态**、**除法异常判断**

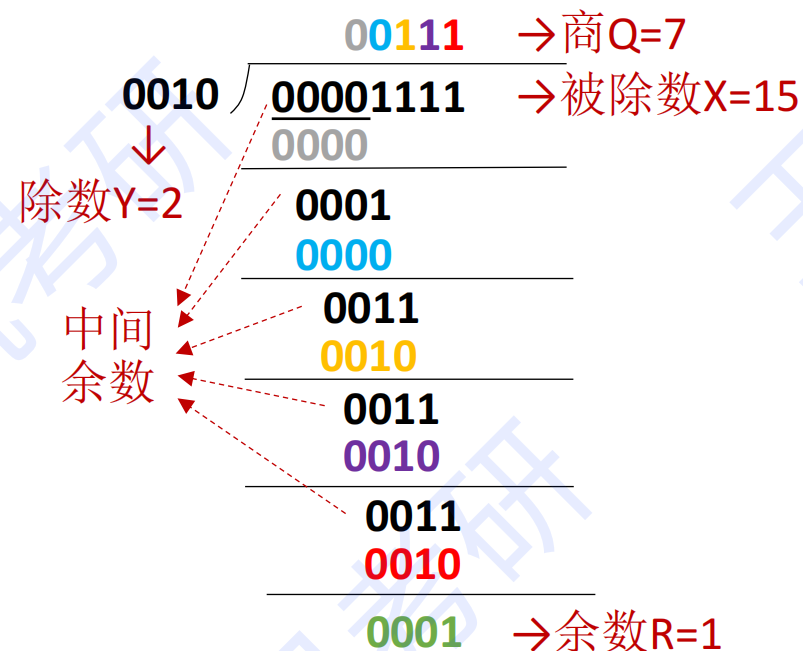
除法电路**运算的中间过程比较复杂**，由于掌握细节需要大量时间，复习性价比不高，因此**建议第一轮复习不要死磕运算的中间过程细节**。可在后几轮复习中，学有余力的情况下再回来抠细节。



手算无符号整数除法（十进制、二进制）



手算方法：“逐位上商，错位相减”



余数的数学定义：被除数 = 商 × 除数 + 余数

十进制上商规则：商 × 除数 的值，要尽可能接近“中间余数”，但又不能大于中间余数

二进制上商规则：商 × 除数 的值，要尽可能接近“中间余数”，但又不能大于中间余数
换句话说，如果 中间余数 ≥ 除数，则上商 1；否则上商 0

无符号整数除法原理：以 4bit 无符号整数为例

若被除数不足 2n bit，需要将
被除数扩展为 2n bit，统一为
2n bit ÷ nbit

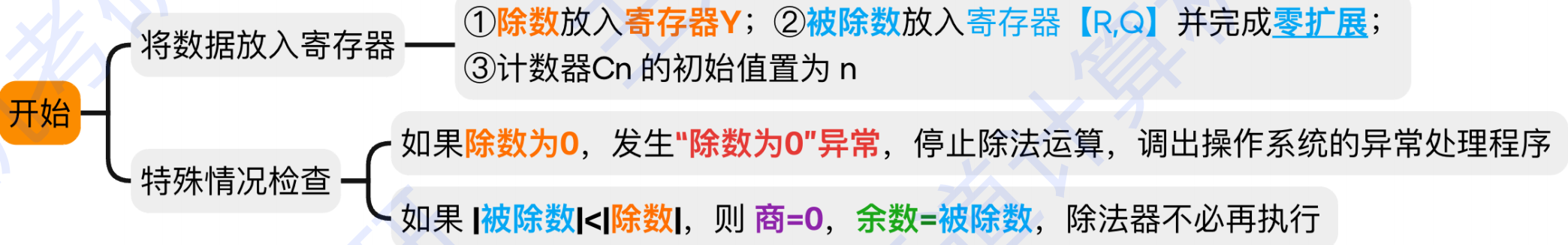
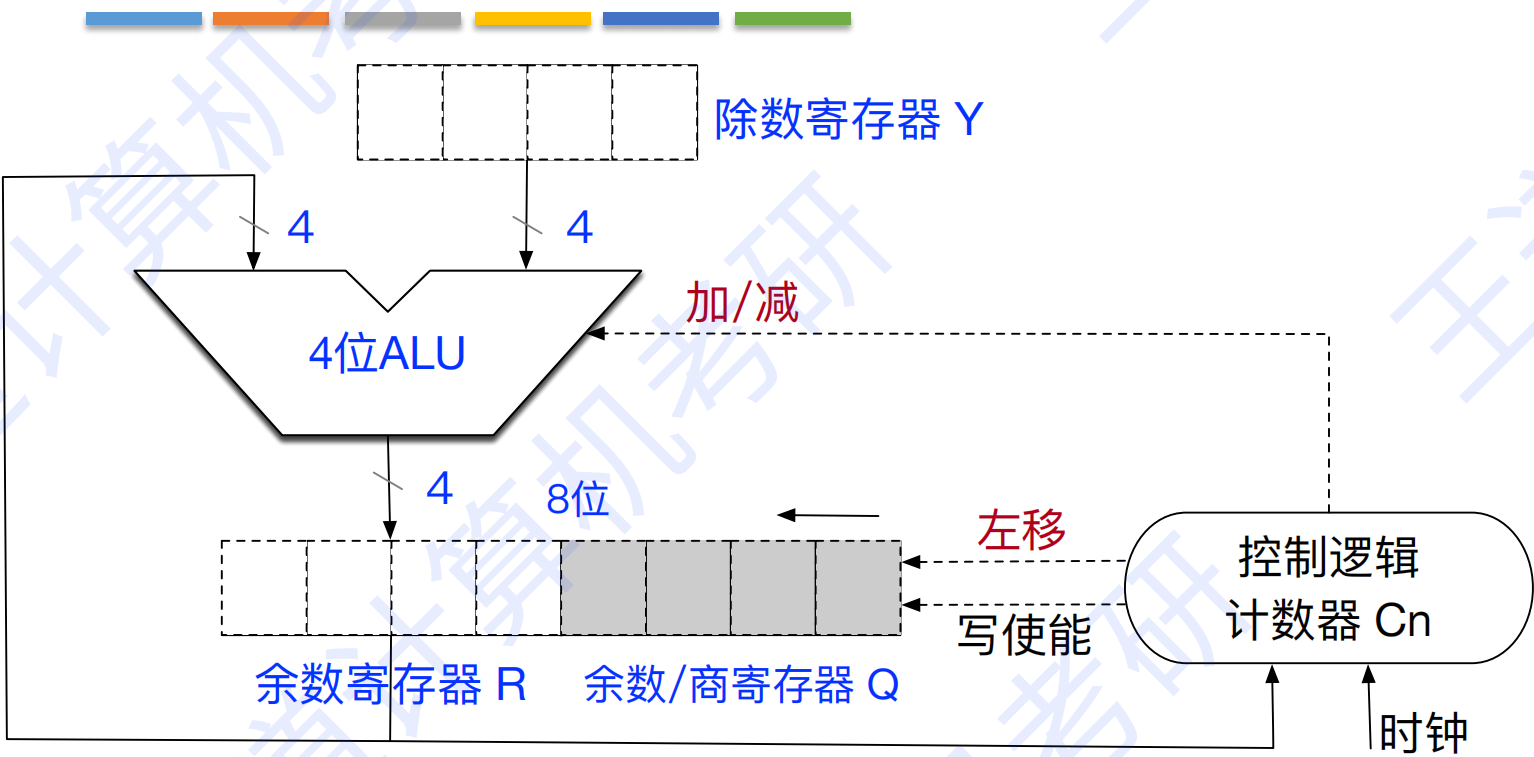
该除法器支持 2n bit ÷ n bit
最终得到 n bit 商、n bit 余数

1111 → 被除数 X=15

0010 → 除数 Y=2

0111 → 商 Q=7

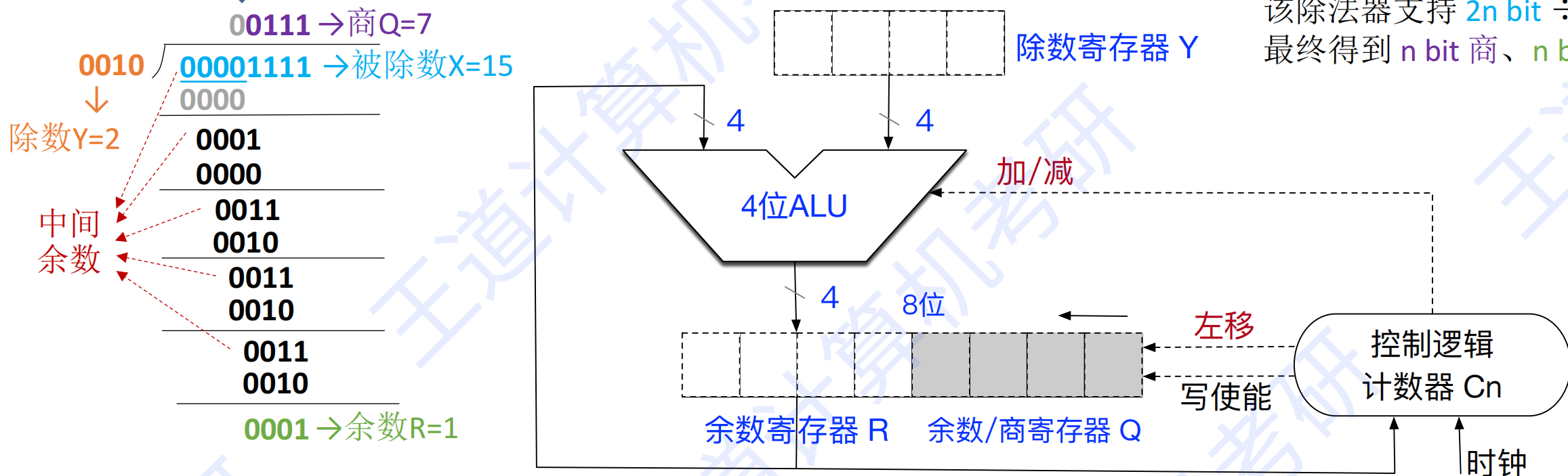
0001 → 余数 R=1



无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 $Cn--$ ，除法运算继续

其余 n 轮处理

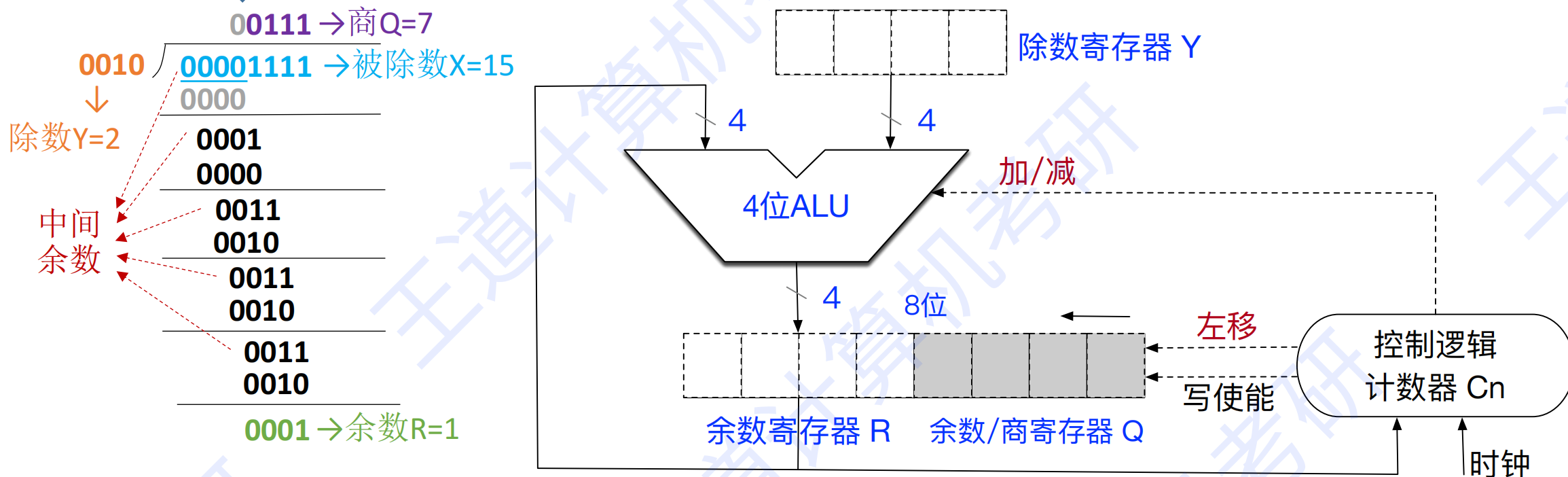
①先左移，空出的位用于上商

②上商，背后的过程可能会进行加法/减法

③计数器 $Cn--$ ，当计数器 $Cn=0$ 时，除法运算结束

第一位商不保存，
仅用于商溢出判断

无符号整数除法原理：以 4bit 无符号整数为例



结束——当计数器Cn=0时，除法运算结束。n bit 寄存器【R】保存余数、n bit 寄存器【Q】保存商

例如：可用该电路实现如下C语言程序的除法运算

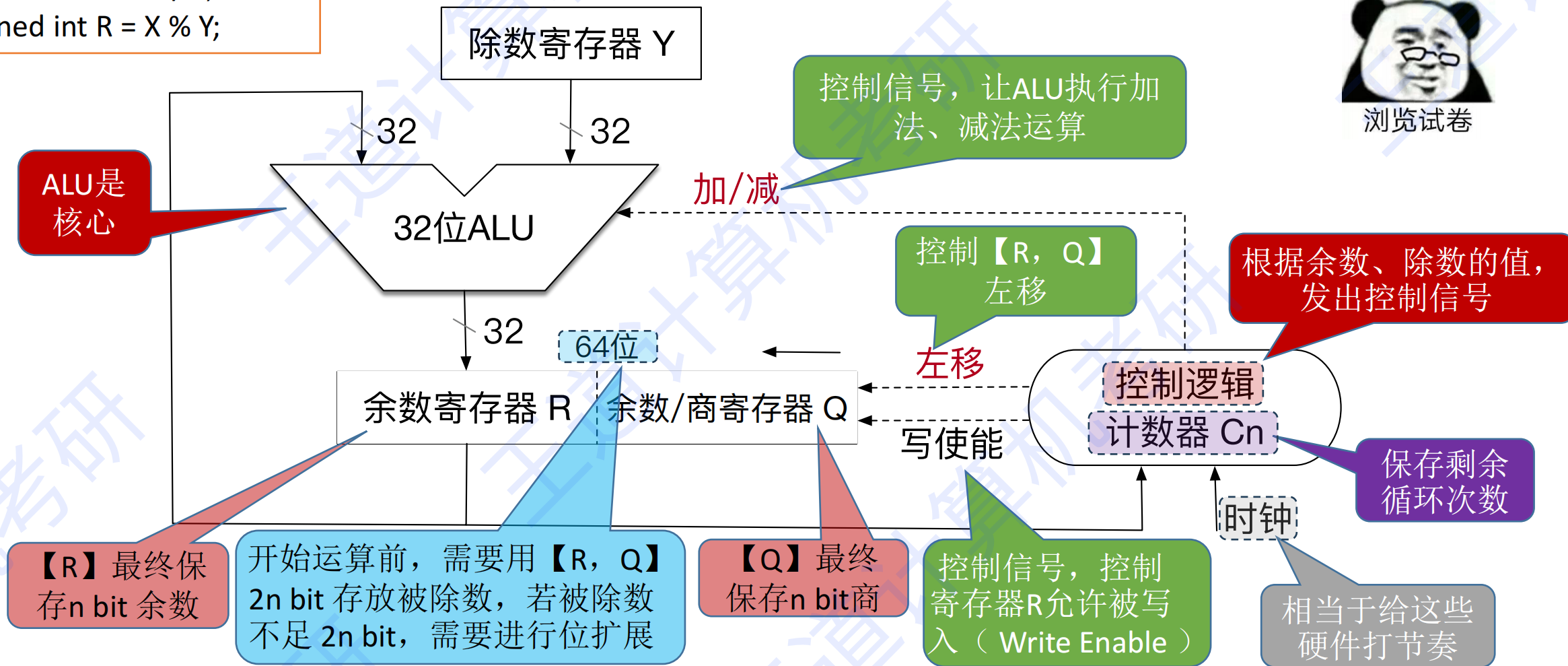
```
unsigned int X=985211;
unsigned int Y=666;
unsigned int Q = X / Y;
unsigned int R = X % Y;
```

32 位无符号数除法运算电路的结构

一种可能得出题思路：各寄存器的初始值、终值分别是多少？



浏览试卷



关于“商溢出”的进一步探讨

该除法器支持

$2n \text{ bit} \div n \text{ bit}$ (双精度除法)

$n \text{ bit} \div n \text{ bit}$ (单精度除法)

需统一为
 $2n \text{ bit} \div n \text{ bit}$

最终都仅保留 **n bit 商**、**n bit 余数**

【例1】双精度除法商溢出

255 / 15 = 17, 余数为0

$$1111\ 1111 / 1111 = 1\ 0001$$

【例2】双精度除法商未溢出

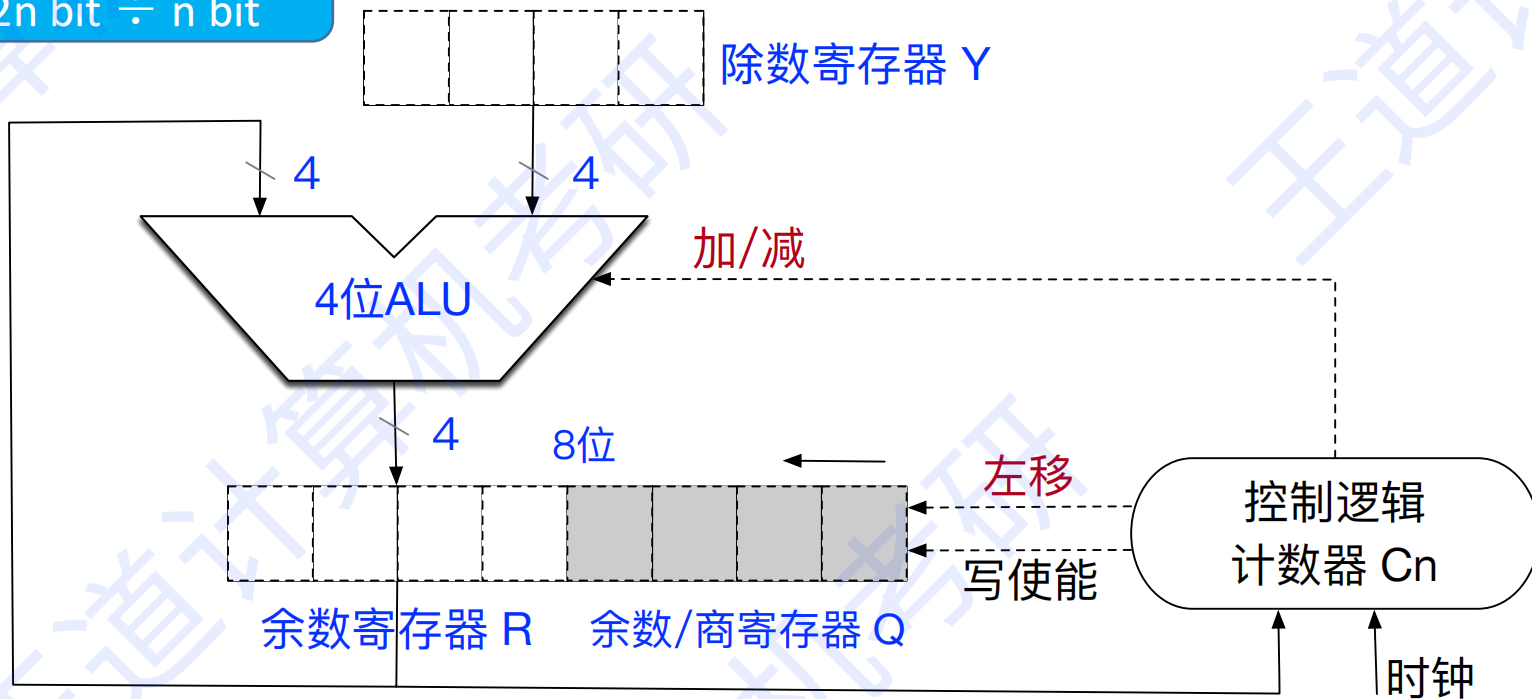
$200/15 = 13$, 余数为5

$$1100\ 1000 / 1111 = 0\ 1101$$

【例3】单精度除法商不可能溢出

15 / 1 = 15, 余数为0

0000 1111/0001 = 1111



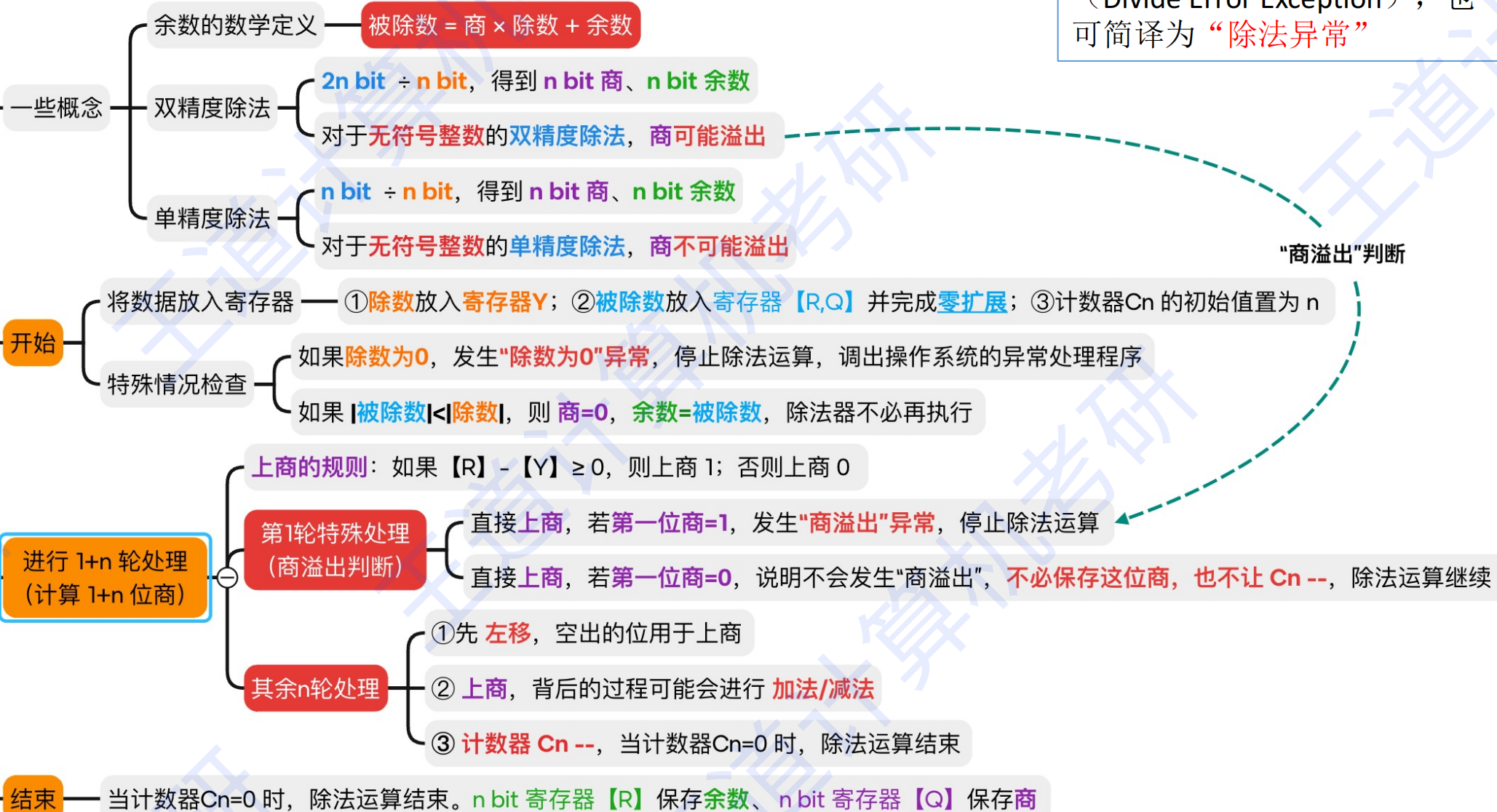
$2n \text{ bit} \div n \text{ bit}$, 商的实际位数有可能超过 $n \text{ bit}$, 因此, 无符号整数的双精度除法可能发生“商溢出”

$n \text{ bit} \div n \text{ bit}$, 商的实际位数不可能超过 $n \text{ bit}$, 因此, 无符号整数单精度除法不可能发生“商溢出”

知识回顾与重要考点

概念补充：在 x86 中，**除数为0**、**商溢出** 都属于“**除法错异常**”（Divide Error Exception），也可简译为“**除法异常**”

无符号整数除法



无符号整数除法原理：以 4bit 无符号整数为例

若被除数不足 2n bit，需要将
被除数扩展为 2n bit，统一为
2n bit ÷ nbit

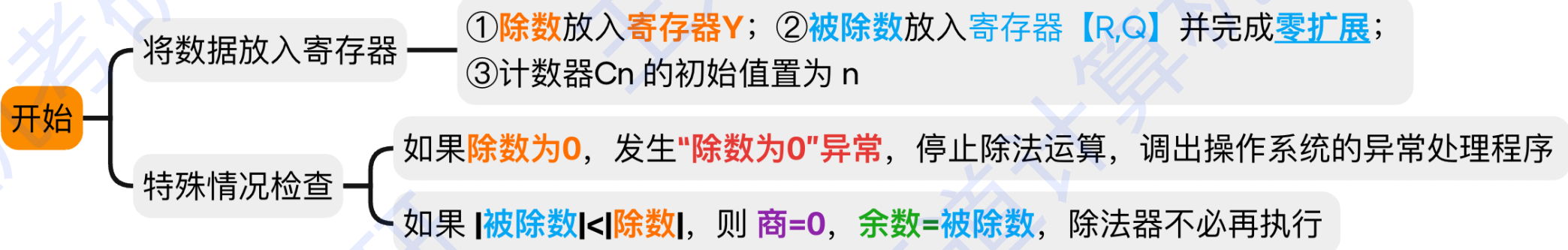
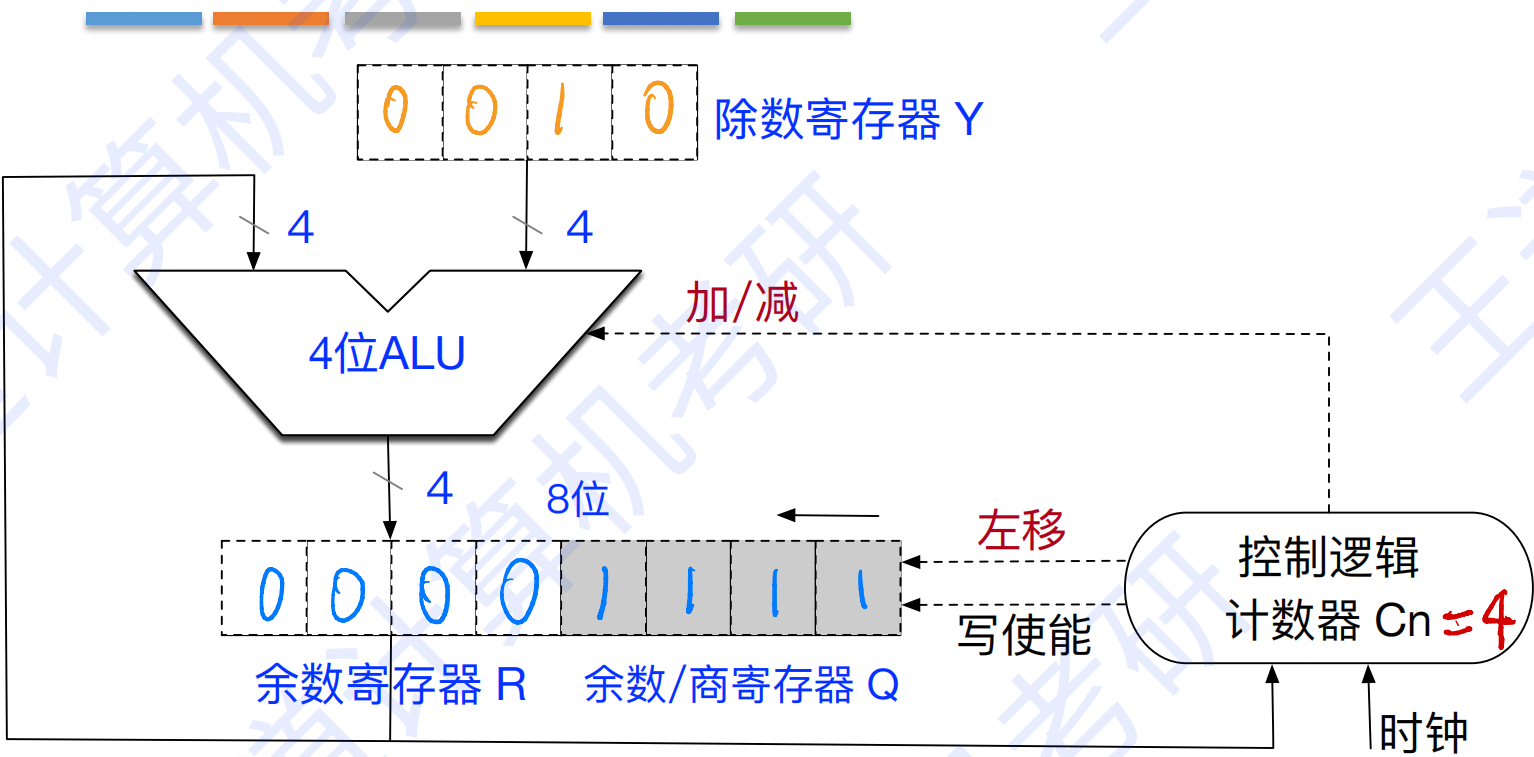
该除法器支持 2n bit ÷ n bit
最终得到 n bit 商、n bit 余数

1111 → 被除数 X=15

0010 → 除数 Y=2

0111 → 商 Q=7

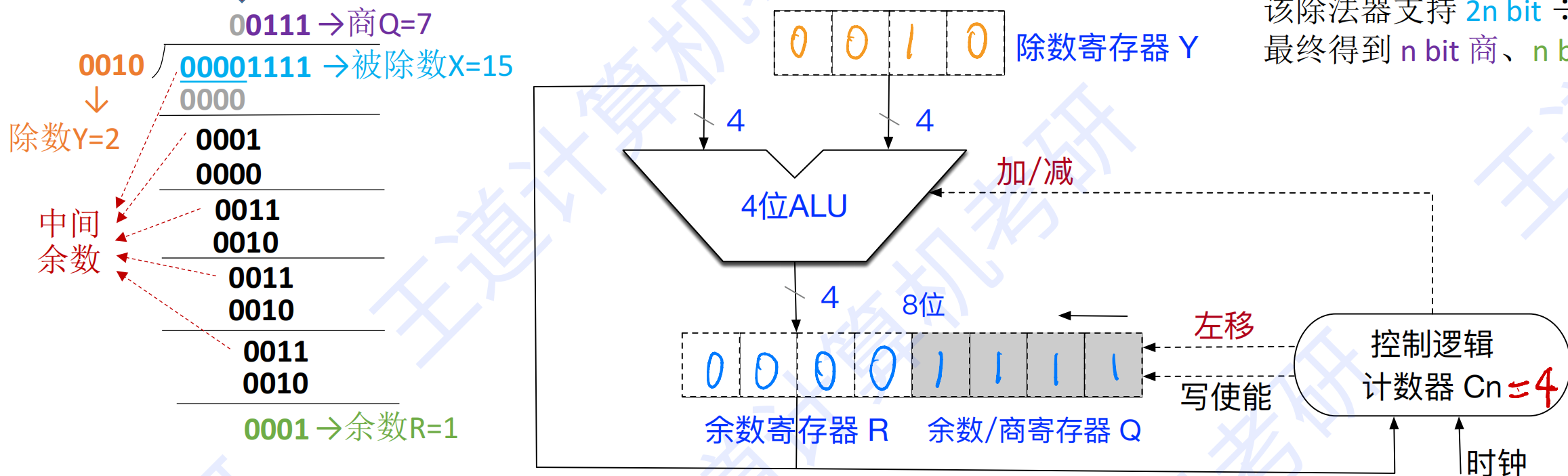
0001 → 余数 R=1



无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 $Cn--$ ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

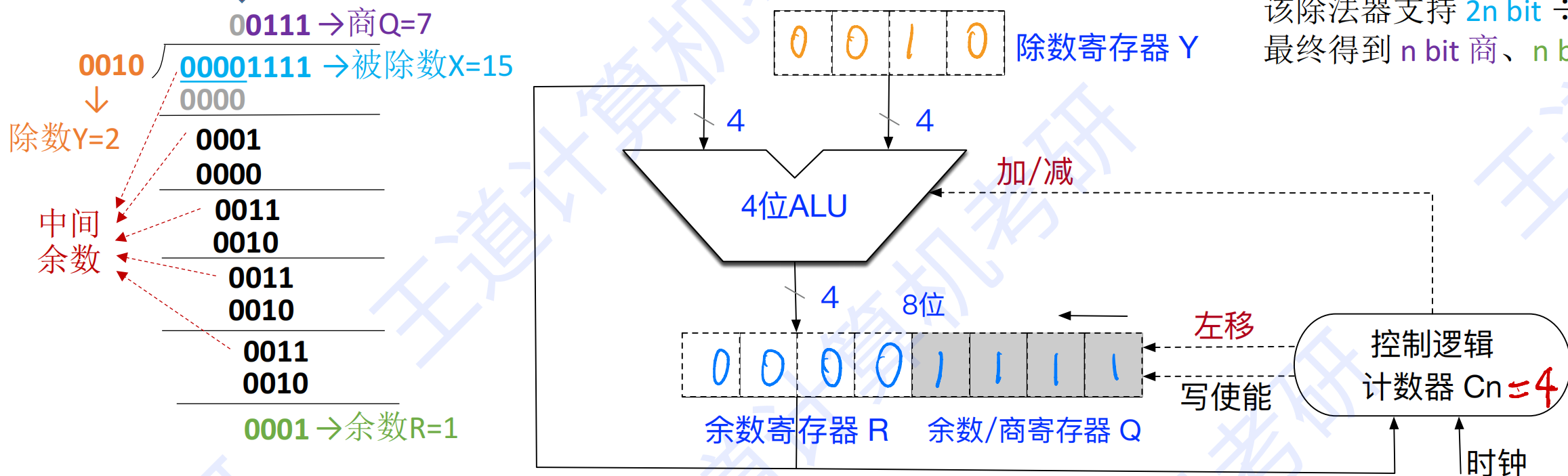
② 上商，背后的过程可能会进行加法/减法

③ 计数器 $Cn--$ ，当计数器 $Cn=0$ 时，除法运算结束

第一位商不保存，
仅用于商溢出判断

无符号整数除法原理：以 4bit 无符号整数为例

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 C_n-- ，除法运算继续

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

其余 n 轮处理

① 先左移，空出的位用于上商

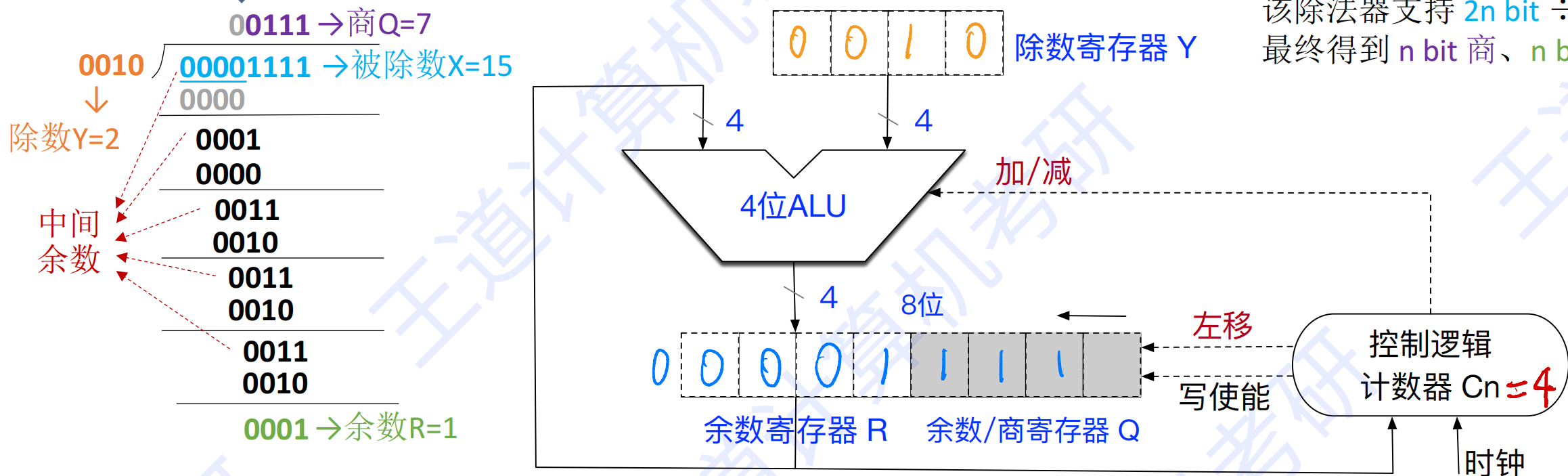
② 上商，背后的过程可能会进行加法/减法

③ 计数器 C_n-- ，当计数器 $C_n=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 $Cn--$ ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

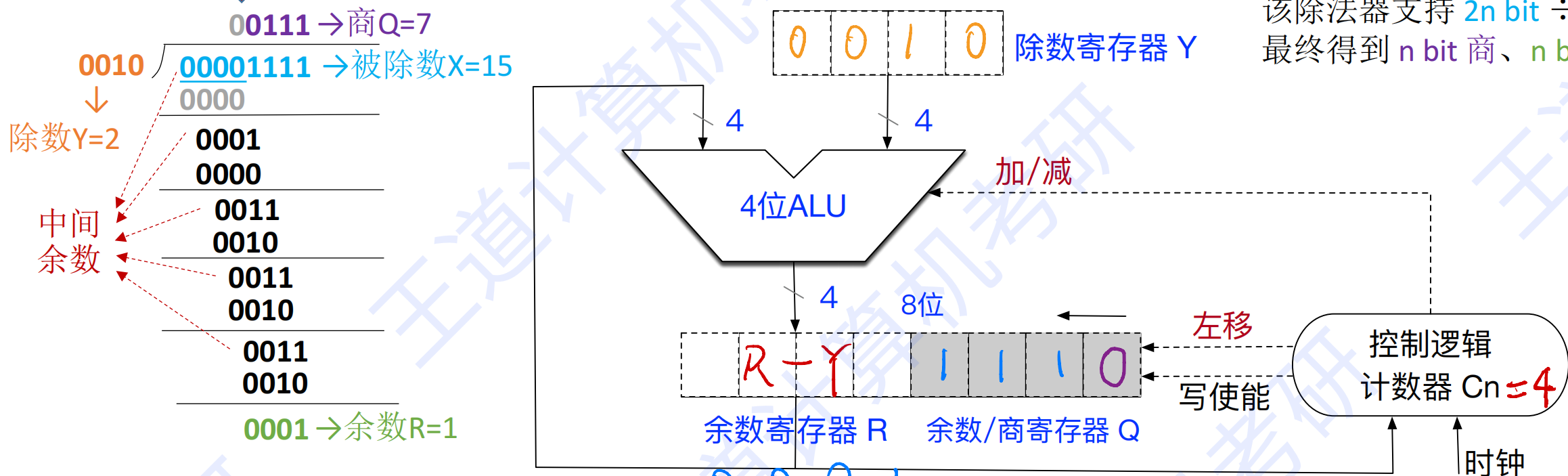
② 上商，背后的过程可能会进行加法/减法

③ 计数器 $Cn--$ ，当计数器 $Cn=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 C_n-- ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

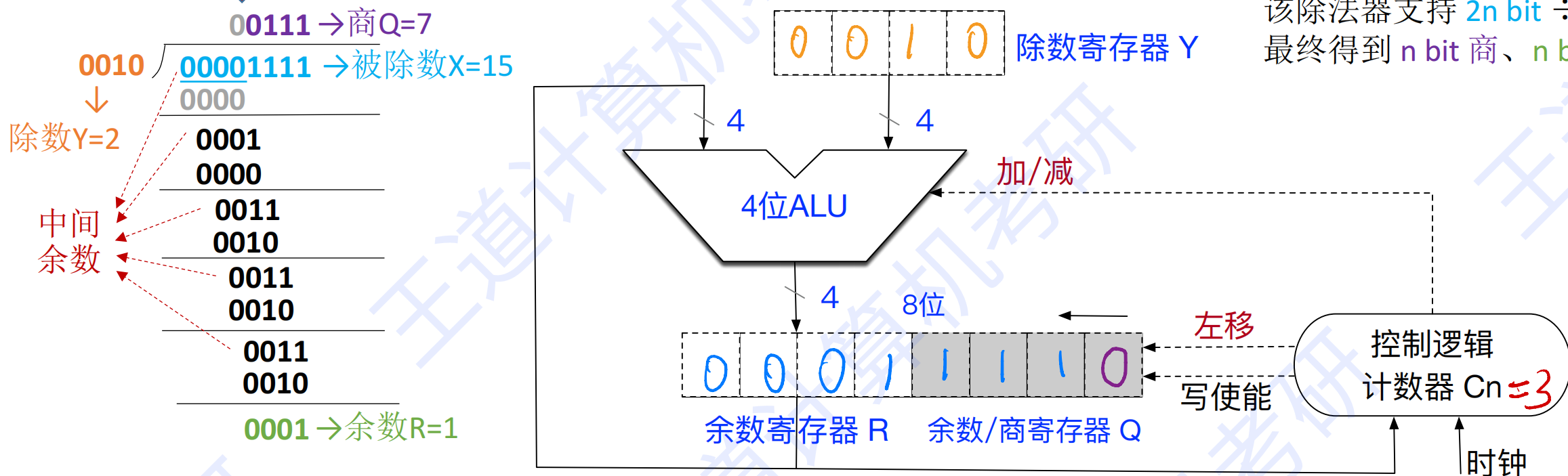
② 上商，背后的过程可能会进行加法/减法

③ 计数器 C_n-- ，当计数器 $C_n=0$ 时，除法运算结束

第一位商不保存，
仅用于商溢出判断

无符号整数除法原理：以 4bit 无符号整数为例

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 $Cn--$ ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

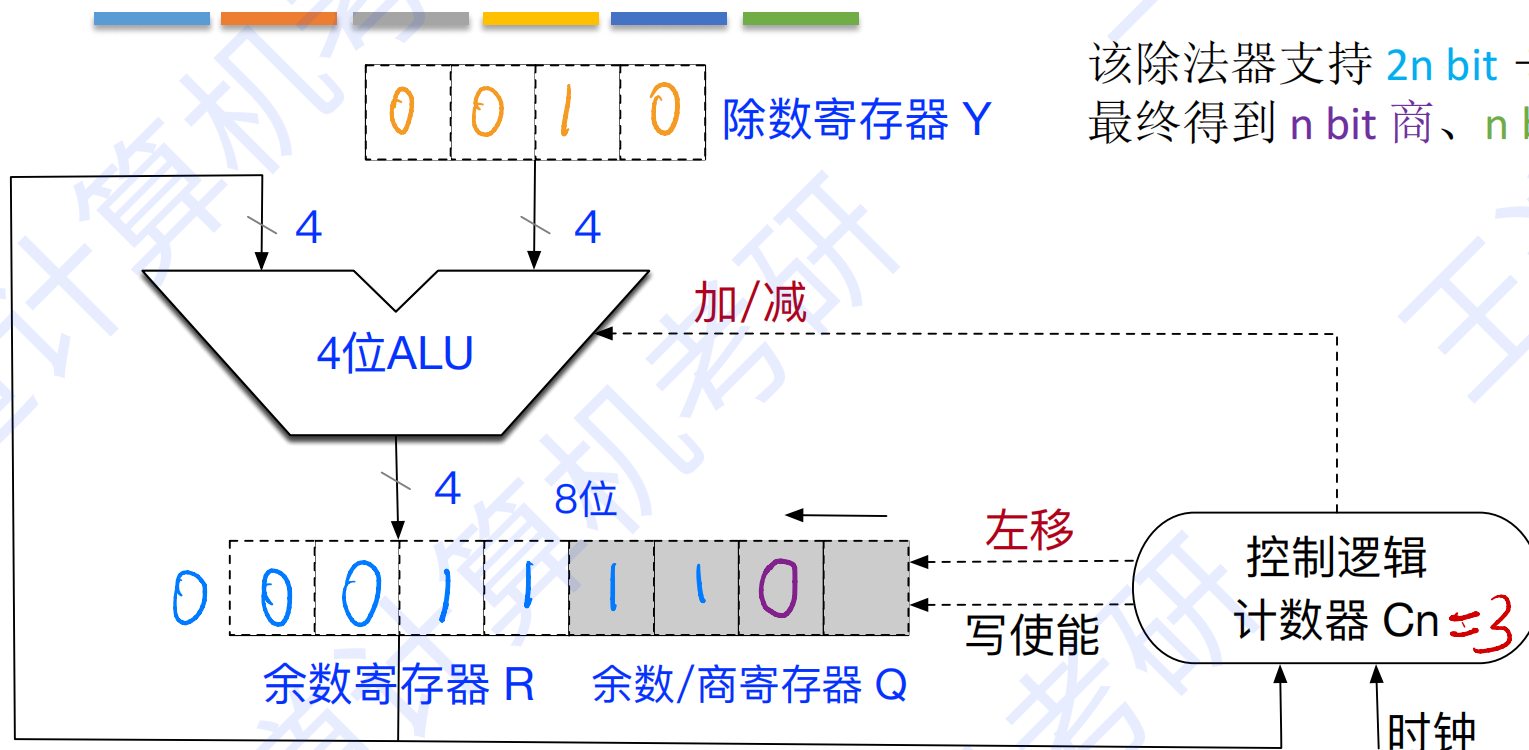
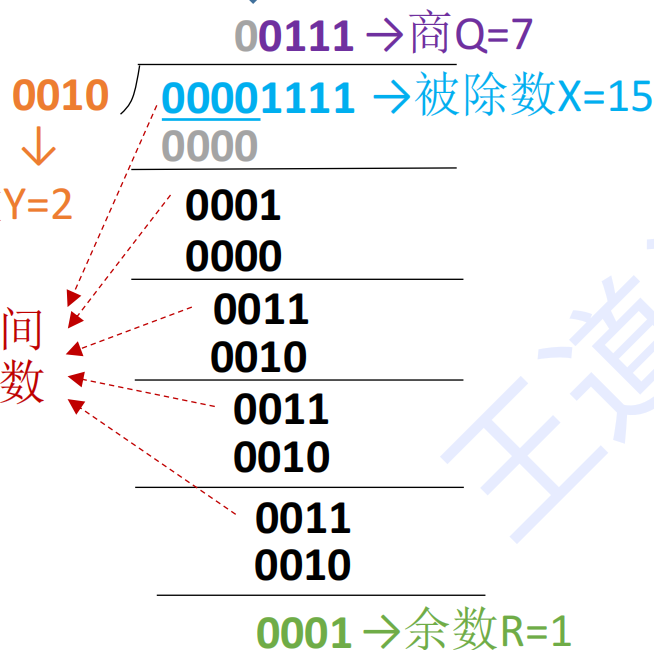
② 上商，背后的过程可能会进行加法/减法

③ 计数器 $Cn--$ ，当计数器 $Cn=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 1+n 轮处理
(计算 1+n 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 Cn --，除法运算继续

其余n轮处理

①先左移，空出的位用于上商

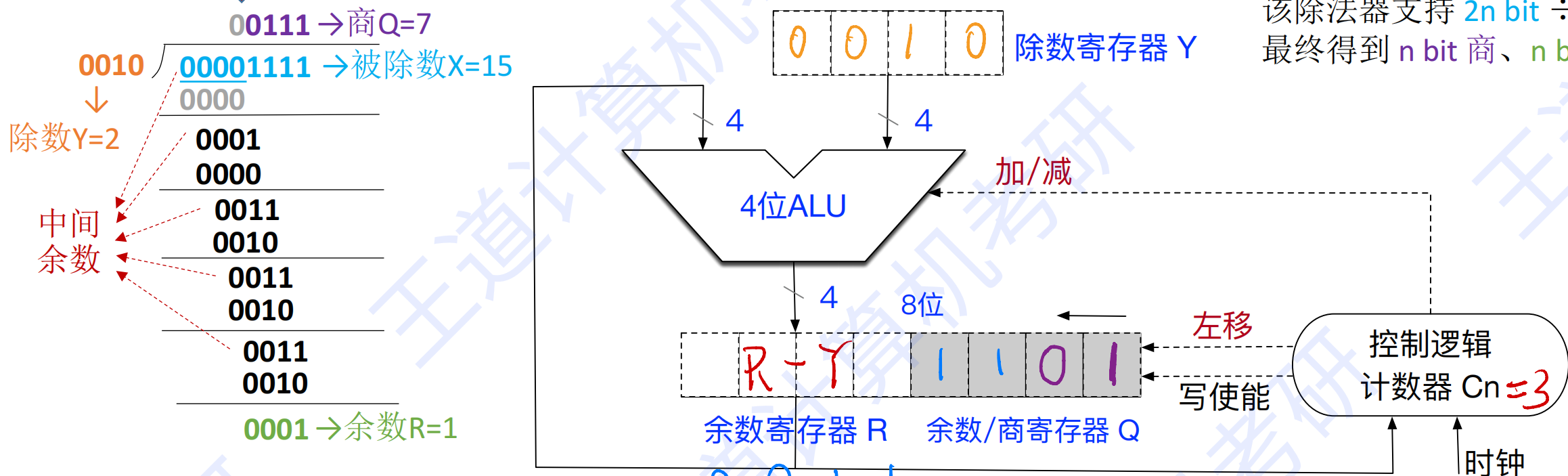
②上商，背后的过程可能会进行加法/减法

③计数器 Cn --，当计数器 Cn=0 时，除法运算结束

第一位商不保存，
仅用于商溢出判断

无符号整数除法原理：以 4bit 无符号整数为例

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 C_n-- ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

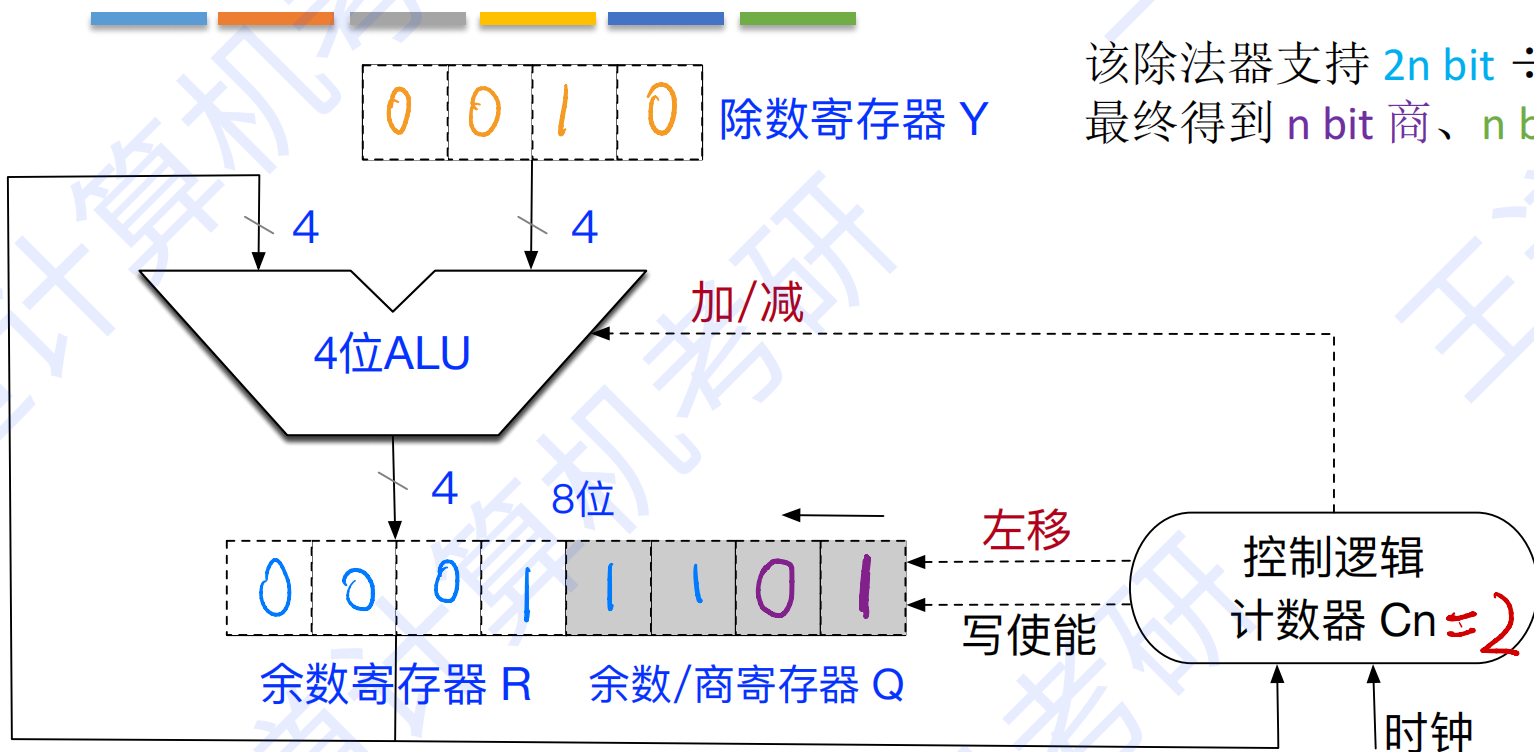
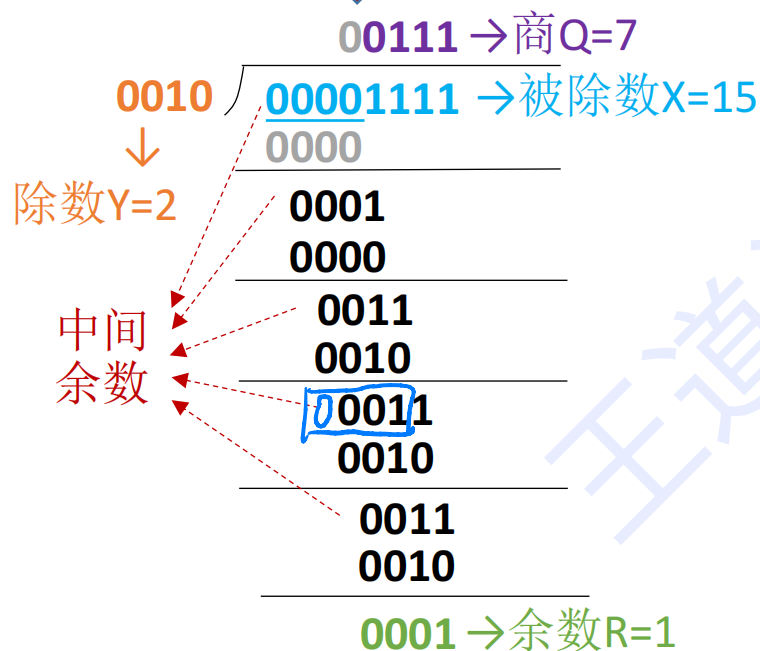
② 上商，背后的过程可能会进行加法/减法

③ 计数器 C_n-- ，当计数器 $C_n=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 C_n-- ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

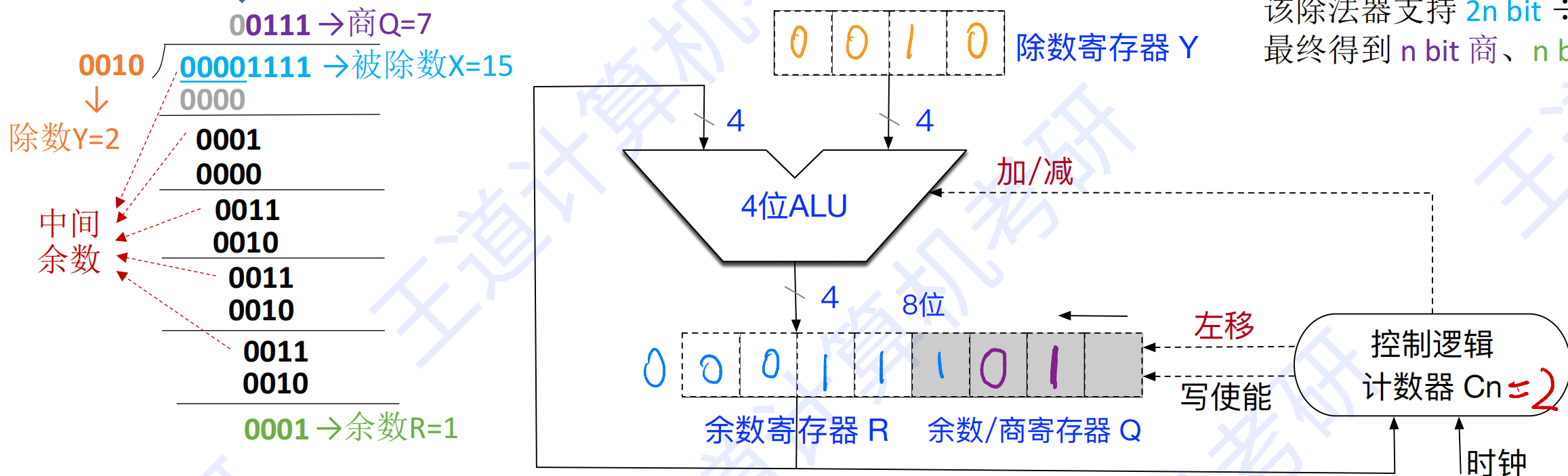
② 上商，背后的过程可能会进行加法/减法

③ 计数器 C_n-- ，当计数器 $C_n=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 $Cn--$ ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

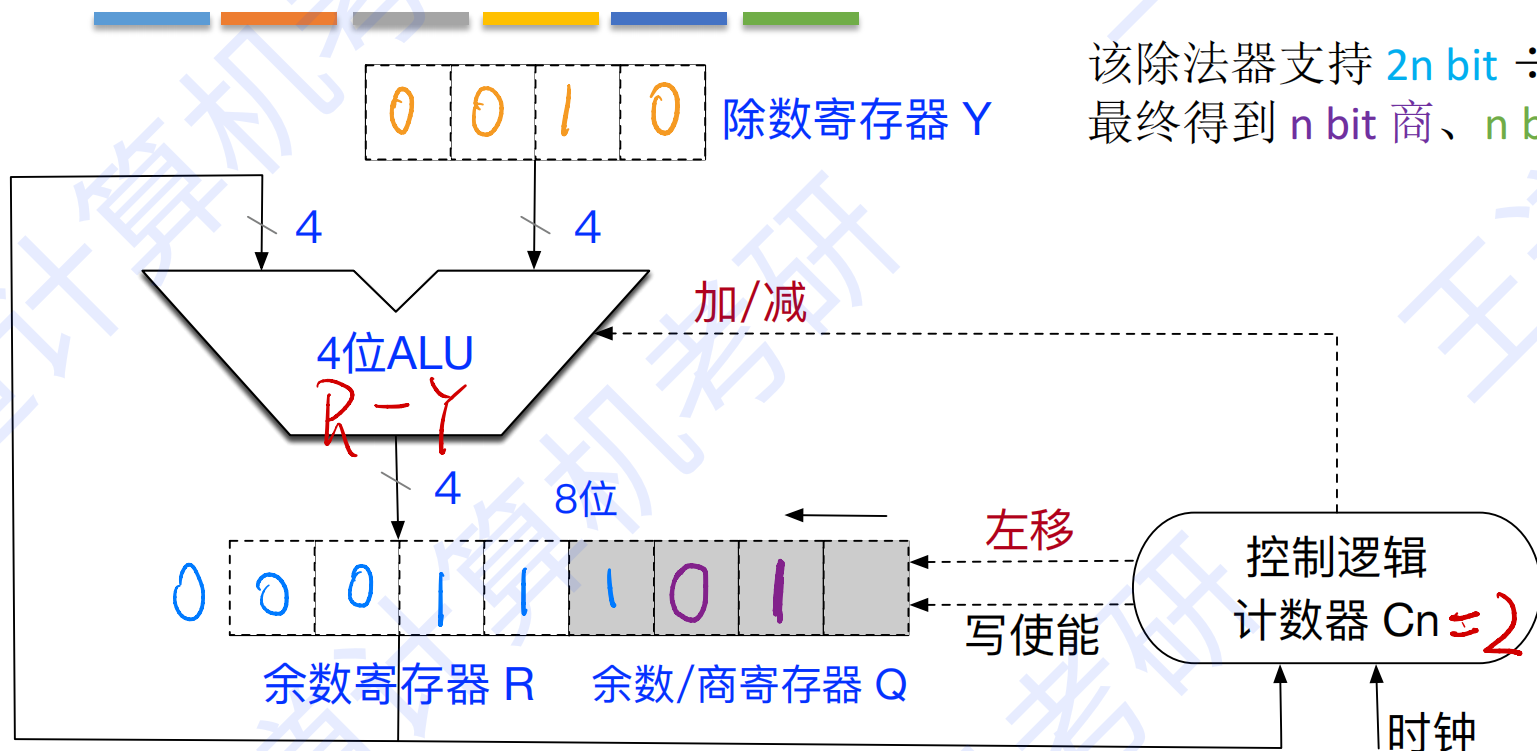
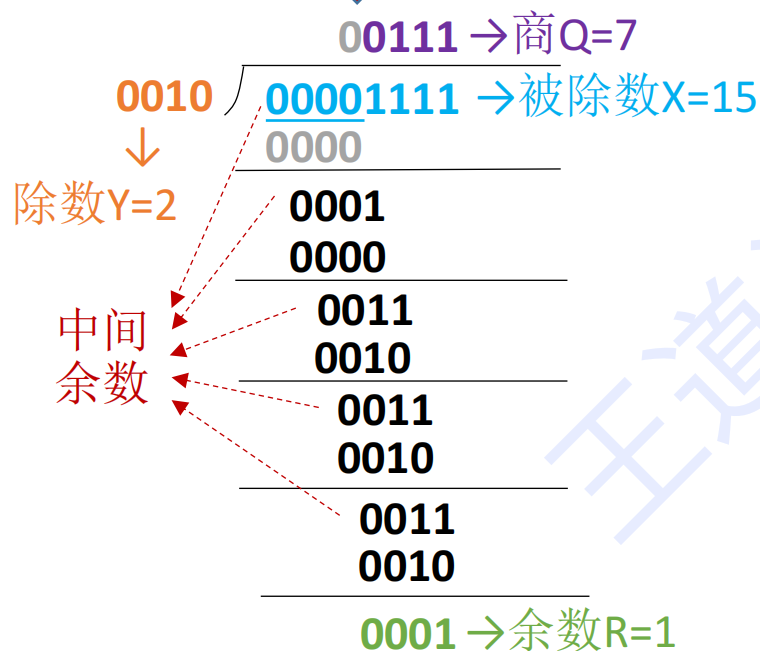
② 上商，背后的过程可能会进行加法/减法

③ 计数器 $Cn--$ ，当计数器 $Cn=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 C_n-- ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

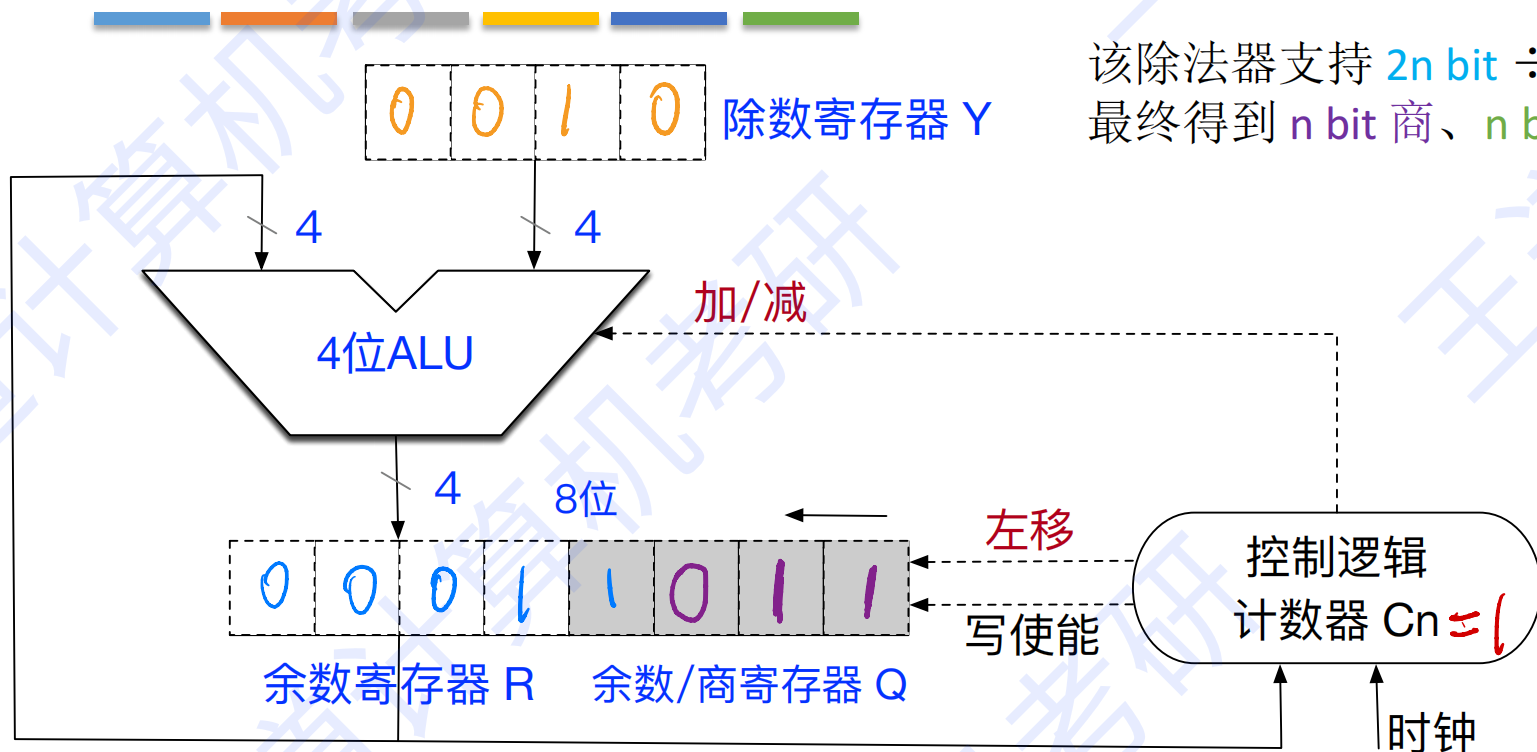
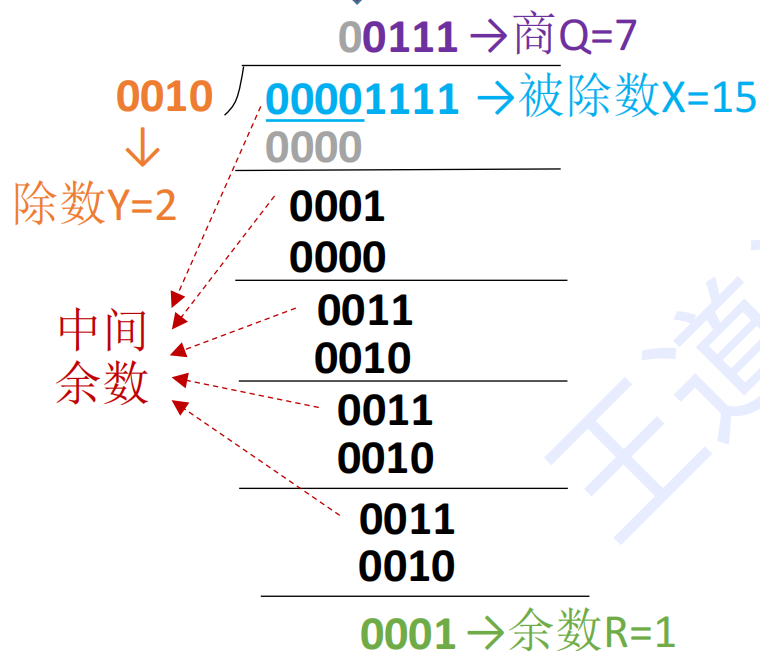
② 上商，背后的过程可能会进行加法/减法

③ 计数器 C_n-- ，当计数器 $C_n=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 $Cn--$ ，除法运算继续

其余 n 轮处理

①先左移，空出的位用于上商

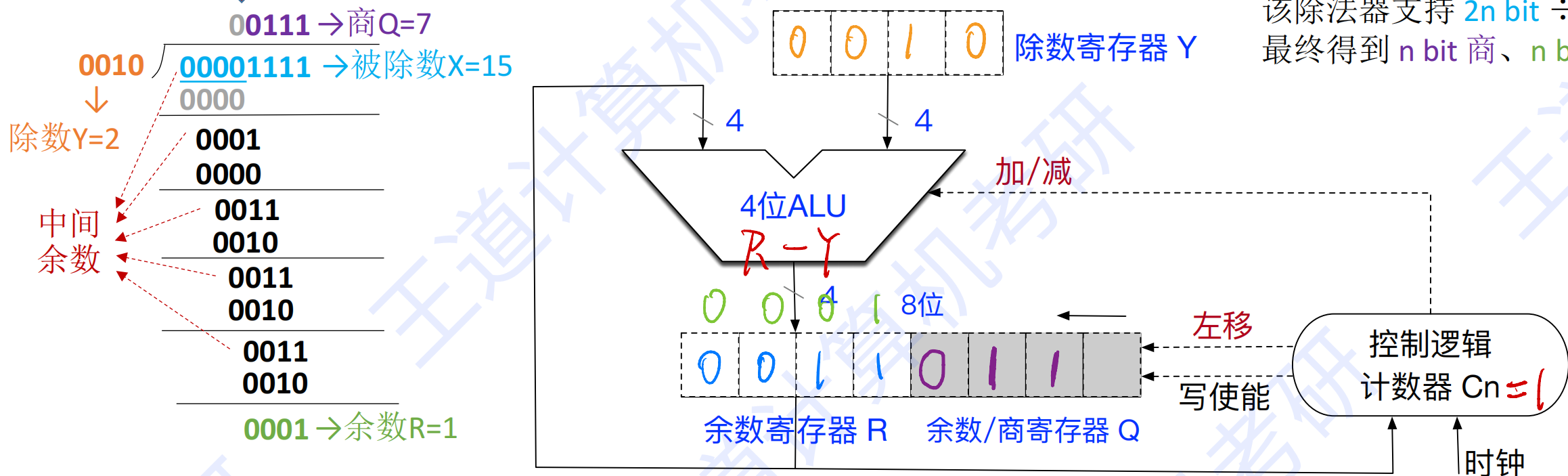
②上商，背后的过程可能会进行加法/减法

③计数器 $Cn--$ ，当计数器 $Cn=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 $C_n --$ ，除法运算继续

其余 n 轮处理

① 先左移，空出的位用于上商

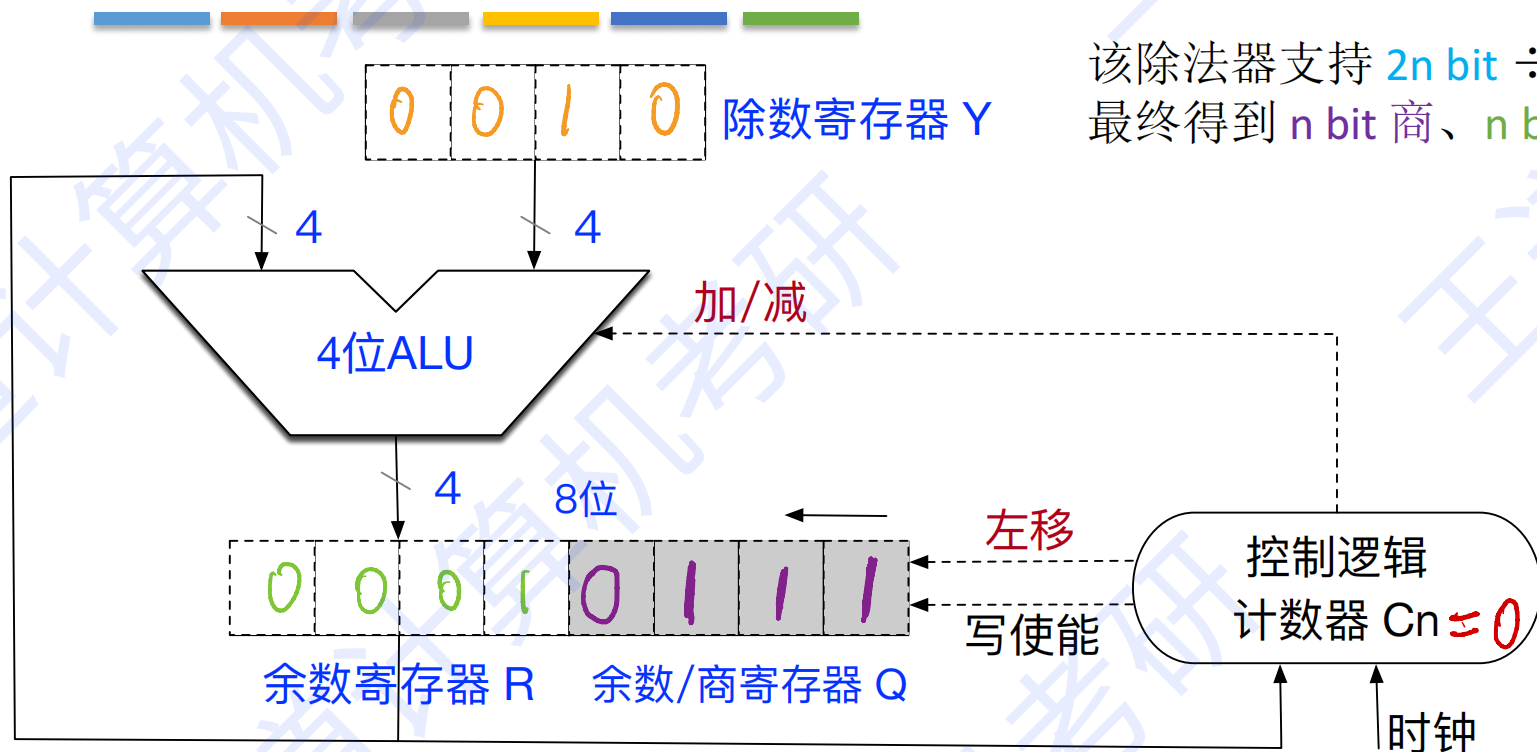
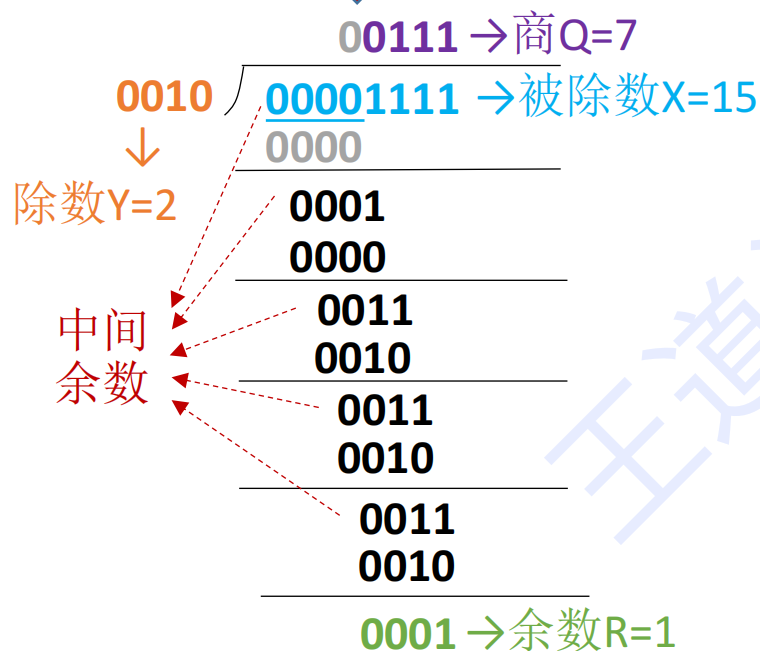
② 上商，背后的过程可能会进行加法/减法

③ 计数器 $C_n --$ ，当计数器 $C_n=0$ 时，除法运算结束

无符号整数除法原理：以 4bit 无符号整数为例

第一位商不保存，
仅用于商溢出判断

该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数



上商的规则：如果 $[R] - [Y] \geq 0$ ，则上商 1；否则上商 0

进行 $1+n$ 轮处理
(计算 $1+n$ 位商)

第1轮特殊处理
(商溢出判断)

直接上商，若第一位商=1，发生“商溢出”异常，停止除法运算

直接上商，若第一位商=0，说明不会发生“商溢出”，不必保存这位商，也不让 C_n-- ，除法运算继续

其余 n 轮处理

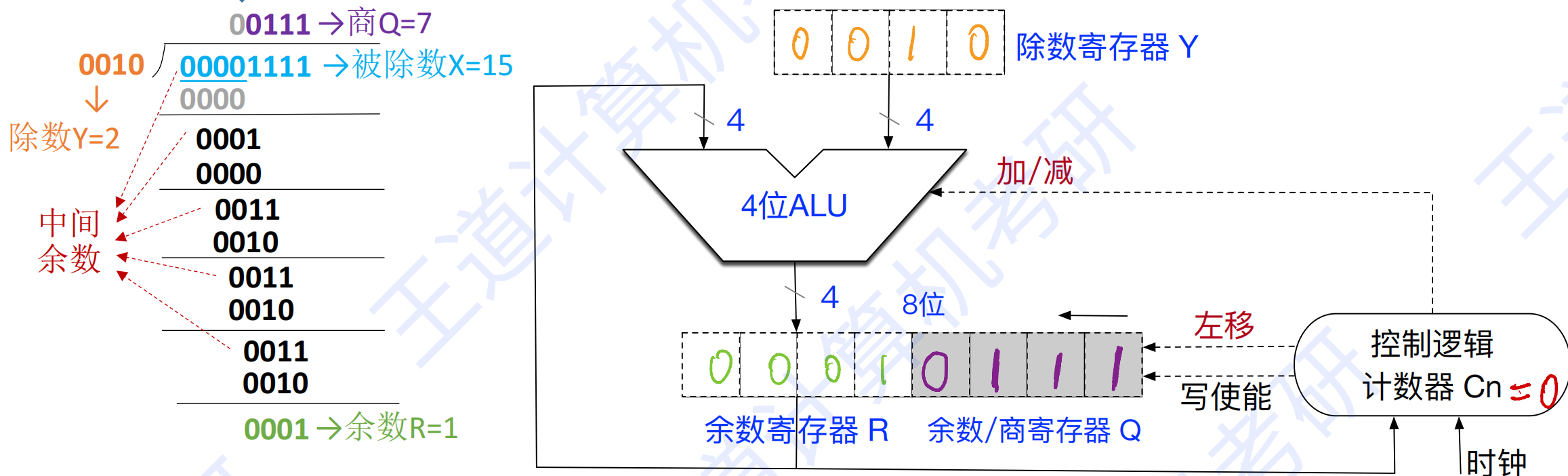
①先左移，空出的位用于上商

②上商，背后的过程可能会进行加法/减法

③计数器 C_n-- ，当计数器 $C_n=0$ 时，除法运算结束

第一位商不保存，
仅用于商溢出判断

无符号整数除法原理：以 4bit 无符号整数为例



结束——当计数器Cn=0时，除法运算结束。 n bit 寄存器【R】保存余数、 n bit 寄存器【Q】保存商